

Movie Ticket Booking System

PL/SQL Procedure Calls, Functions & Verification Demo Script (Oracle 21c)

STAKEHOLDER 1: CUSTOMER BOOKING LIFECYCLE

1. REGISTER CUSTOMER

PROCEDURE / FUNCTION CALL:

```
ACCEPT name CHAR PROMPT 'Enter Customer Name: '  
ACCEPT email CHAR PROMPT 'Enter Email: '  
ACCEPT mobile CHAR PROMPT 'Enter Mobile: '  
ACCEPT password CHAR PROMPT 'Enter Password: '  
  
BEGIN  
REGISTER_CUSTOMER('&name;', '&email;', '&mobile;', '&password;');  
END;  
/
```

INPUT VALUES:

Name	Vijay
Email	vijay@gmail.com
Mobile	90000000004
Password	1234

VERIFICATION QUERY:

```
SELECT CUSTOMER_ID, NAME, EMAIL, MOBILE, ROLE  
FROM CUSTOMERS  
WHERE EMAIL='&email;';
```

SAY TO MAM:

"The customer is registered successfully. I am verifying the newly inserted customer record in the CUSTOMERS table."

2. LOGIN CUSTOMER

PROCEDURE / FUNCTION CALL:

```
ACCEPT email CHAR PROMPT 'Enter Email: '
ACCEPT password CHAR PROMPT 'Enter Password: '

BEGIN
LOGIN_CUSTOMER('&email;', '&password;');
END;
/
```

INPUT VALUES:

Email	vijay@gmail.com
Password	1234

SAY TO MAM:

"This procedure checks whether the entered email and password are valid using an implicit cursor and outputs the customer ID and role."

3. SELECT LOCATION

PROCEDURE / FUNCTION CALL:

```
BEGIN  
  SELECT_LOCATION;  
END;  
/
```

INPUT VALUES:

Note

No input required. Uses explicit cursor to display all active cinema locations.

SAY TO MAM:

"This displays all available cinema locations using an explicit cursor loop across the LOCATIONS table."

4. SELECT MOVIE

PROCEDURE / FUNCTION CALL:

```
ACCEPT loc_id NUMBER PROMPT 'Enter Location ID: '  
  
BEGIN  
  SELECT_MOVIE(&loc_id);  
END;  
/
```

INPUT VALUES:

Location ID

2 (Puducherry)

SAY TO MAM:

"I selected location 2, which is Puducherry. Now the movies available in this location are displayed by joining SHOWS, SCREENS, and THEATRES."

5. SELECT THEATRE

PROCEDURE / FUNCTION CALL:

```
ACCEPT loc_id NUMBER PROMPT 'Enter Location ID: '  
ACCEPT movie_id NUMBER PROMPT 'Enter Movie ID: '  
  
BEGIN  
SELECT_THEATRE(&loc;_id, &movie;_id);  
END;  
/
```

INPUT VALUES:

Location ID	2 (Puducherry)
Movie ID	1 (Baththa)

SAY TO MAM:

"Now I am selecting the theatre exhibiting the chosen movie in Puducherry using an explicit cursor."

6. SELECT SHOW

PROCEDURE / FUNCTION CALL:

```
ACCEPT movie_id NUMBER PROMPT 'Enter Movie ID: '  
ACCEPT theatre_id NUMBER PROMPT 'Enter Theatre ID: '  
ACCEPT show_date CHAR PROMPT 'Enter Show Date (YYYY-MM-DD): '  
  
BEGIN  
SELECT_SHOW(  
    &movie;_id,  
    &theatre;_id,  
    TO_DATE('&show;_date', 'YYYY-MM-DD')  
);  
END;  
/
```

INPUT VALUES:

Movie ID	1 (Baththa)
Theatre ID	2 (PVR INOX, White Town)
Show Date	2026-10-05

SAY TO MAM:

"This displays the scheduled shows available for the selected movie, theatre, and date including screen name, time, and base price."

7. VIEW AVAILABLE SEATS

PROCEDURE / FUNCTION CALL:

```
ACCEPT show_id NUMBER PROMPT 'Enter Show ID: '  
  
BEGIN  
VIEW_AVAILABLE_SEATS(&show;_id);  
END;  
/
```

INPUT VALUES:

Show ID	501 (or 504)
---------	--------------

Important: Check the output and confirm that the seat you want to book is shown with STATUS = 'AVAILABLE'.

SAY TO MAM:

"Before booking, I am checking the available seats for this show. The cursor displays seat row, seat number, and ticket price."

8. CALCULATE TICKET PRICE — PL/SQL FUNCTION

PROCEDURE / FUNCTION CALL:

```
ACCEPT show_id NUMBER PROMPT 'Enter Show ID: '  
ACCEPT seat_count NUMBER PROMPT 'Enter Number of Seats: '  
  
SELECT CALCULATE_TICKET_PRICE(&show;_id, &seat;_count) AS TOTAL_TICKET_PRICE  
FROM DUAL;
```

INPUT VALUES:

Show ID	501
Number of Seats	2

SAY TO MAM:

"This PL/SQL function calculates the total ticket price by fetching the base price from the SHOWS table and adding convenience fees of Rs 30 per seat."

9. APPLY COUPON — PL/SQL FUNCTION

PROCEDURE / FUNCTION CALL:

```
ACCEPT coupon_code CHAR PROMPT 'Enter Coupon Code: '  
ACCEPT amount NUMBER PROMPT 'Enter Ticket Amount: '  
  
SELECT APPLY_COUPON('&coupon_code', &amount;) AS DISCOUNTED_AMOUNT  
FROM DUAL;
```

INPUT VALUES:

Coupon Code	WELCOME100
Ticket Amount	560

SAY TO MAM:

"This function validates the coupon against usage limits, expiry, and minimum order, applies the discount, and increments the coupon USED_COUNT."

10. VIEW FOOD MENU

PROCEDURE / FUNCTION CALL:

```
BEGIN  
VIEW_FOOD_MENU;  
END;  
/
```

INPUT VALUES:

Note	No input required. Lists all available F&B; popcorn, beverages, and combo packs.
------	---

SAY TO MAM:

"This procedure displays the available cinema concession snacks and beverages with item ID, category, and unit price."

11. CREATE BOOKING

PROCEDURE / FUNCTION CALL:

```
ACCEPT customer_id NUMBER PROMPT 'Enter Customer ID: '  
ACCEPT show_id NUMBER PROMPT 'Enter Show ID: '  
ACCEPT show_seat_id NUMBER PROMPT 'Enter Show Seat ID: '  
  
BEGIN  
CREATE_BOOKING(  
  &customer_id,  
  &show_id,  
  &show_seat_id  
);  
END;  
/
```

INPUT VALUES:

Customer ID	1001
Show ID	501
Show Seat ID	1 (Seat A1)

Important: Trigger TRG_BOOK_SEAT automatically fires after insertion, updating the show seat status to BOOKED and generating a ticket record.

VERIFICATION QUERY:

```
SELECT BOOKING_ID, CUSTOMER_ID, SHOW_ID, STATUS, TOTAL_AMOUNT  
FROM BOOKINGS  
WHERE CUSTOMER_ID = &customer_id  
ORDER BY BOOKING_ID DESC;
```

SAY TO MAM:

"The booking has been created. The AFTER INSERT trigger TRG_BOOK_SEAT marks the seat as BOOKED and generates the initial ticket record."

12. ORDER FOOD CONCESSIONS

PROCEDURE / FUNCTION CALL:

```
ACCEPT booking_id NUMBER PROMPT 'Enter Booking ID: '
ACCEPT food_id NUMBER PROMPT 'Enter Food ID: '
ACCEPT quantity NUMBER PROMPT 'Enter Quantity: '

BEGIN
ORDER_FOOD(
&booking;_id,
&food;_id,
&quantity;
);
END;
/
```

INPUT VALUES:

Booking ID	5001
Food ID	602 (Large Popcorn)
Quantity	1

VERIFICATION QUERY:

```
SELECT FOOD_ORDER_ID, BOOKING_ID, FOOD_ID, QUANTITY, SUBTOTAL
FROM FOOD_ORDERS
WHERE BOOKING_ID = &booking;_id;
```

SAY TO MAM:

"The food order is inserted for this booking. The procedure calculates the subtotal from the FOOD table price and attaches it to the booking."

13. PROCESS PAYMENT

PROCEDURE / FUNCTION CALL:

```
ACCEPT booking_id NUMBER PROMPT 'Enter Booking ID: '  
ACCEPT method CHAR PROMPT 'Enter Payment Method (UPI/CARD/NET_BANKING): '  
ACCEPT amount NUMBER PROMPT 'Enter Payment Amount: '  
  
BEGIN  
PROCESS_PAYMENT(  
  &booking_id,  
  '&method;',  
  &amount;  
);  
END;  
/
```

INPUT VALUES:

Booking ID	5001
Payment Method	UPI
Payment Amount	620

VERIFICATION QUERY:

```
SELECT PAYMENT_ID, BOOKING_ID, PAYMENT_METHOD, AMOUNT, PAYMENT_STATUS  
FROM PAYMENTS  
WHERE BOOKING_ID = &booking_id;  
  
-- Verify Booking Status Updated by Trigger:  
SELECT BOOKING_ID, STATUS, FINAL_AMOUNT  
FROM BOOKINGS  
WHERE BOOKING_ID = &booking_id;
```

SAY TO MAM:

"The payment record has been inserted. Trigger TRG_PAYMENT_CONFIRM automatically changes the booking status from PENDING to CONFIRMED."

14. GENERATE E-TICKET

PROCEDURE / FUNCTION CALL:

```
ACCEPT booking_id NUMBER PROMPT 'Enter Booking ID: '  
  
BEGIN  
GENERATE_TICKET(&booking_id);  
END;  
/
```

INPUT VALUES:

Booking ID	5001
------------	------

VERIFICATION QUERY:

```
SELECT TICKET_ID, BOOKING_ID, SHOW_SEAT_ID, STATUS, GENERATED_AT  
FROM TICKETS  
WHERE BOOKING_ID = &booking_id;
```

SAY TO MAM:

"The e-ticket is generated by joining 8 relational tables (BOOKINGS, SHOWS, MOVIES, THEATRES, SCREENS, TICKETS, SHOW_SEATS, SEATS) with movie, theatre, showtime, seat, and price."

15. VIEW BOOKING HISTORY

PROCEDURE / FUNCTION CALL:

```
ACCEPT customer_id NUMBER PROMPT 'Enter Customer ID: '  
  
BEGIN  
VIEW_BOOKING_HISTORY(&customer;_id);  
END;  
/
```

INPUT VALUES:

Customer ID	1001
-------------	------

SAY TO MAM:

"This displays the complete booking history of the customer including past and confirmed reservations."

16. CANCEL BOOKING

PROCEDURE / FUNCTION CALL:

```
ACCEPT booking_id NUMBER PROMPT 'Enter Booking ID: '  
ACCEPT customer_id NUMBER PROMPT 'Enter Customer ID: '  
  
BEGIN  
CANCEL_BOOKING(  
  &booking;_id,  
  &customer;_id  
);  
END;  
/
```

INPUT VALUES:

Booking ID	5001
Customer ID	1001

VERIFICATION QUERY:

```
SELECT BOOKING_ID, CUSTOMER_ID, STATUS, FINAL_AMOUNT  
FROM BOOKINGS  
WHERE BOOKING_ID = &booking;_id;
```

SAY TO MAM:

"The customer cancelled the reservation. Trigger TRG_RELEASE_SEAT automatically fires to release the seat, cancel the ticket, and initiate a refund."

17. VERIFY SEAT RELEASE TRIGGER

PROCEDURE / FUNCTION CALL:

```
-- Check seat status after booking cancellation:
ACCEPT show_seat_id NUMBER PROMPT 'Enter Show Seat ID: '

SELECT SHOW_SEAT_ID, SHOW_ID, STATUS
FROM SHOW_SEATS
WHERE SHOW_SEAT_ID = &show;_seat_id;
```

INPUT VALUES:

Show Seat ID	1 (Seat A1)
--------------	-------------

Important: Trigger TRG_RELEASE_SEAT automatically updates STATUS back to 'AVAILABLE' upon cancellation.

SAY TO MAM:

"When the booking was cancelled, the trigger automatically released the seat and changed its status back to AVAILABLE for other customers."

18. VIEW REFUND STATUS

PROCEDURE / FUNCTION CALL:

```
ACCEPT booking_id NUMBER PROMPT 'Enter Booking ID: '

BEGIN
VIEW_REFUND_STATUS(&booking;_id);
END;
/
```

INPUT VALUES:

Booking ID	5001
------------	------

VERIFICATION QUERY:

```
SELECT REFUND_ID, BOOKING_ID, AMOUNT, STATUS, REFUND_DATE
FROM REFUNDS
WHERE BOOKING_ID = &booking;_id;
```

SAY TO MAM:

"This procedure displays the refund record created automatically by TRG_RELEASE_SEAT with status INITIATED."

19. ADD MOVIE REVIEW

PROCEDURE / FUNCTION CALL:

```
ACCEPT customer_id NUMBER PROMPT 'Enter Customer ID: '  
ACCEPT movie_id NUMBER PROMPT 'Enter Movie ID: '  
ACCEPT rating NUMBER PROMPT 'Enter Rating (1-5): '  
ACCEPT review_text CHAR PROMPT 'Enter Review Text: '  
  
BEGIN  
ADD_REVIEW(  
  &customer_id,  
  &movie_id,  
  &rating,  
  '&review_text'  
);  
END;  
/
```

INPUT VALUES:

Customer ID	1001
Movie ID	1 (Baththa)
Rating	5
Review Text	Blockbuster action drama with outstanding music!

VERIFICATION QUERY:

```
SELECT REVIEW_ID, CUSTOMER_ID, MOVIE_ID, RATING, TO_CHAR(REVIEW_TEXT) AS REVIEW  
FROM REVIEWS  
WHERE CUSTOMER_ID = &customer_id AND MOVIE_ID = &movie_id;
```

SAY TO MAM:

"The customer review has been inserted and I am verifying it in the REVIEWS table. It prevents duplicate reviews per user for the same movie."

STAKEHOLDER 2: THEATRE MANAGER OPERATIONS

20. THEATRE MANAGER — ONBOARD / REGISTER THEATRE

PROCEDURE / FUNCTION CALL:

```
ACCEPT theatre_id NUMBER PROMPT 'Enter Theatre ID: '  
ACCEPT name CHAR PROMPT 'Enter Theatre Name: '  
ACCEPT location_id NUMBER PROMPT 'Enter Location ID: '  
ACCEPT address CHAR PROMPT 'Enter Address: '  
ACCEPT phone CHAR PROMPT 'Enter Phone: '  
ACCEPT email CHAR PROMPT 'Enter Email: '  
ACCEPT manager_id NUMBER PROMPT 'Enter Manager Customer ID: '  
  
BEGIN  
REGISTER_THEATRE(  
  &theatre_id,  
  '&name;',  
  &location_id,  
  '&address;',  
  '&phone;',  
  '&email;',  
  &manager_id  
);  
END;  
/
```

INPUT VALUES:

Theatre ID	209
Name	PVR Heritage Pondy
Location ID	2 (Puducherry)
Address	Mission Street, White Town, Puducherry
Phone	0413-2223344
Email	pondy@pvr.com
Manager Customer ID	1002

Important: Newly registered theatres start with IS_VERIFIED = 0 until platform admin verifies licenses and safety clearance.

VERIFICATION QUERY:

```
SELECT THEATRE_ID, NAME, LOCATION_ID, IS_VERIFIED, IS_ACTIVE  
FROM THEATRES  
WHERE THEATRE_ID = &theatre_id;
```

SAY TO MAM:

"The theatre owner registers their cinema property with contact details. The system initializes it with IS_VERIFIED = 0 pending admin document verification."

21. THEATRE MANAGER — VIEW THEATRE BOOKINGS

PROCEDURE / FUNCTION CALL:

```
ACCEPT theatre_id NUMBER PROMPT 'Enter Theatre ID: '

BEGIN
VIEW_THEATRE_BOOKINGS(&theatre;_id);
END;
/
```

INPUT VALUES:

Theatre ID	2 (PVR INOX, White Town)
------------	--------------------------

SAY TO MAM:
"This procedure allows the theatre manager to view all customer bookings, showtimes, seats, and gross revenue specific to their own theatre."

22. THEATRE MANAGER — AUDIT LIVE SEAT OCCUPANCY

PROCEDURE / FUNCTION CALL:

```
ACCEPT show_id NUMBER PROMPT 'Enter Show ID: '  
  
BEGIN  
VIEW_AVAILABLE_SEATS(&show;_id);  
END;  
/
```

INPUT VALUES:

Show ID	501
---------	-----

SAY TO MAM:

"The duty manager audits live screen occupancy and inspects available vs booked seats before admitting patrons at the turnstile gate."

23. THEATRE MANAGER — CONCESSIONS & FOOD INVENTORY

PROCEDURE / FUNCTION CALL:

```
BEGIN  
VIEW_FOOD_MENU;  
END;  
/
```

INPUT VALUES:

Note	No input required. Lists active concession snacks, drinks, combos, and prices.
------	--

SAY TO MAM:

"This displays the cafeteria concession menu and prices for theatre staff to verify food order fulfillment."

24. THEATRE MANAGER — VERIFY TURNSTILE TICKET AT GATE

PROCEDURE / FUNCTION CALL:

```
ACCEPT booking_id NUMBER PROMPT 'Enter Booking ID to Scan: '  
  
BEGIN  
GENERATE_TICKET(&booking_id);  
END;  
/
```

INPUT VALUES:

Booking ID	5001 (or Scanned QR ID)
------------	-------------------------

VERIFICATION QUERY:

```
SELECT TICKET_ID, BOOKING_ID, STATUS, GENERATED_AT  
FROM TICKETS  
WHERE BOOKING_ID = &booking_id;
```

SAY TO MAM:

"When the customer arrives at the cinema gate, the duty manager scans the QR code or inputs the Booking ID to verify the movie title, auditorium screen, seat numbers, and ticket count before punching admission."

STAKEHOLDER 3: CINEMABOOK PLATFORM ADMIN GOVERNANCE

25. ADMIN — ADD MOVIE

PROCEDURE / FUNCTION CALL:

```
ACCEPT movie_id NUMBER PROMPT 'Enter Movie ID: '
ACCEPT movie_name CHAR PROMPT 'Enter Movie Name: '
ACCEPT language CHAR PROMPT 'Enter Language: '
ACCEPT genre CHAR PROMPT 'Enter Genre: '
ACCEPT duration NUMBER PROMPT 'Enter Duration in Minutes: '

BEGIN
ADD_MOVIE(
&movie;_id,
'&movie;_name',
'&language;',
'&genre;',
&duration;
);
END;
/
```

INPUT VALUES:

Movie ID	107
Movie Name	Retro
Language	Tamil
Genre	Drama
Duration	160 mins

VERIFICATION QUERY:

```
SELECT MOVIE_ID, TITLE, LANGUAGE, GENRE, DURATION_MINUTES, STATUS
FROM MOVIES
WHERE MOVIE_ID = &movie;_id;
```

SAY TO MAM:
"The platform admin adds a new movie release. The procedure initially creates it with UPCOMING status in the MOVIES table."

26. ADMIN — UPDATE MOVIE

PROCEDURE / FUNCTION CALL:

```
ACCEPT movie_id NUMBER PROMPT 'Enter Movie ID: '  
ACCEPT movie_name CHAR PROMPT 'Enter Updated Movie Name: '  
ACCEPT status CHAR PROMPT 'Enter Status (RUNNING/UPCOMING/COMPLETED): '  
  
BEGIN  
UPDATE_MOVIE(  
  &movie_id,  
  '&movie_name',  
  '&status';  
);  
END;  
/
```

INPUT VALUES:

Movie ID	107
Updated Name	Retro - Special Edition
Status	RUNNING

VERIFICATION QUERY:

```
SELECT MOVIE_ID, TITLE, STATUS  
FROM MOVIES  
WHERE MOVIE_ID = &movie_id;
```

SAY TO MAM:

"The movie details and release status have been updated. Customers can now book tickets since the status is RUNNING."

27. ADMIN — VERIFY PARTNER THEATRE

PROCEDURE / FUNCTION CALL:

```
ACCEPT theatre_id NUMBER PROMPT 'Enter Theatre ID to Verify: '  
  
BEGIN  
  VERIFY_THEATRE(&theatre_id);  
END;  
/
```

INPUT VALUES:

Theatre ID	209
------------	-----

VERIFICATION QUERY:

```
SELECT THEATRE_ID, NAME, IS_VERIFIED, IS_ACTIVE  
FROM THEATRES  
WHERE THEATRE_ID = &theatre_id;
```

SAY TO MAM:

"CinemaBook admin verifies the theatre's commercial license and fire safety certificate, changing IS_VERIFIED to 1 so the cinema can host active shows."

28. ADMIN — CREATE SHOW & AUTO-SEAT GENERATION

PROCEDURE / FUNCTION CALL:

```
ACCEPT show_id NUMBER PROMPT 'Enter Show ID: '
ACCEPT movie_id NUMBER PROMPT 'Enter Movie ID: '
ACCEPT screen_id NUMBER PROMPT 'Enter Screen ID: '
ACCEPT show_date CHAR PROMPT 'Enter Show Date (YYYY-MM-DD): '
ACCEPT show_time CHAR PROMPT 'Enter Show Time: '
ACCEPT price NUMBER PROMPT 'Enter Base Price: '

BEGIN
CREATE_SHOW(
  &show_id,
  &movie_id,
  &screen_id,
  TO_DATE('&show_date', 'YYYY-MM-DD'),
  '&show_time',
  &price;
);
END;
/
```

INPUT VALUES:

Show ID	507
Movie ID	1 (Baththa)
Screen ID	1
Show Date	2026-10-10
Show Time	07:30 PM
Base Price	200

Important: CREATE_SHOW automatically generates SHOW_SEATS records for all physical seats in that screen with calculated seat tier pricing.

VERIFICATION QUERY:

```
SELECT SHOW_ID, MOVIE_ID, SCREEN_ID, SHOW_DATE, SHOW_TIME, BASE_PRICE
FROM SHOWS
WHERE SHOW_ID = &show_id;

-- Verify Auto-Generated Seats:
SELECT COUNT(*) AS SEATS_GENERATED
FROM SHOW_SEATS
WHERE SHOW_ID = &show_id;
```

SAY TO MAM:

"The admin creates a show. The procedure automatically creates SHOW_SEATS records for all seats in the screen, ready for customer reservations."

29. ADMIN — PLATFORM BOOKING & REVENUE REPORT

PROCEDURE / FUNCTION CALL:

```
BEGIN
VIEW_BOOKING_REPORT;
END;
/
```

INPUT VALUES:

Note	No input required. Aggregates platform bookings, occupancy, and total revenue.
------	--

SAY TO MAM:

"This procedure displays platform-wide analytics using aggregation cursors across BOOKINGS to report total ticket bookings and gross revenue."

30. ADMIN — REMOVE / DEACTIVATE MOVIE

PROCEDURE / FUNCTION CALL:

```
ACCEPT movie_id NUMBER PROMPT 'Enter Movie ID to Remove: '

BEGIN
REMOVE_MOVIE(&movie;_id);
END;
/
```

INPUT VALUES:

Movie ID	107
----------	-----

VERIFICATION QUERY:

```
SELECT MOVIE_ID, TITLE, STATUS
FROM MOVIES
WHERE MOVIE_ID = &movie;_id;
```

SAY TO MAM:

"The admin soft-deletes the movie by updating its STATUS to CANCELLED, maintaining referential integrity for existing bookings."